

SEMANA 3 ACADEMIA MONTERREY

Despliegue de una aplicación Spring Boot en AWS EC2

En esta actividad se realizó el despliegue de una aplicación desarrollada con Spring Boot en un servidor virtual de Amazon Web Services (AWS), utilizando el servicio Amazon EC2.

El objetivo principal fue conocer el proceso básico necesario para llevar una aplicación Java desde un entorno local hasta un servidor en la nube, incluyendo la configuración de acceso, conexión remota mediante SSH, preparación del entorno, generación del archivo ejecutable `.jar`, transferencia del artefacto al servidor y ejecución de la aplicación.

El proceso realizado fue el siguiente:

1. Creación y configuración de una cuenta/usuario IAM.
2. Creación de las credenciales necesarias para trabajar con AWS.
3. Creación de una instancia EC2.
4. Generación y descarga de una llave privada `.pem`.
5. Configuración del acceso mediante SSH.
6. Conexión al servidor EC2 desde Git Bash.
7. Verificación de los recursos disponibles en el servidor.
8. Instalación de Java 21.
9. Compilación y empaquetado de la aplicación Maven en el equipo local.
10. Transferencia del archivo `.jar` hacia EC2 mediante SCP.
11. Verificación de la integridad del archivo transferido.
12. Ejecución de la aplicación Spring Boot en el servidor.
13. Acceso a Swagger desde un navegador mediante la IP pública de la instancia.
14. Prueba de autenticación y creación de una tarea mediante la API.

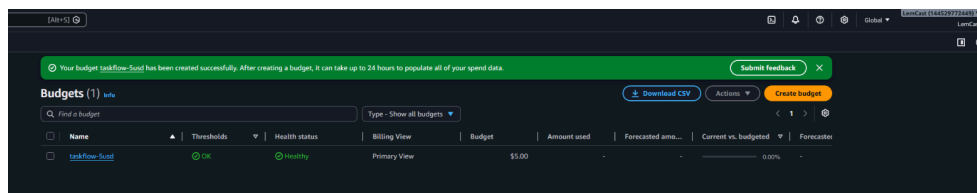
2. Configuración inicial de AWS

2.1 Creación del budget y el usuario IAM

Como primer paso se configuró el acceso a AWS mediante un usuario de IAM (Identity and Access Management).

IAM permite administrar usuarios, credenciales y permisos dentro de AWS. En lugar de utilizar directamente la cuenta principal de AWS para las actividades, se creó un usuario destinado a trabajar con los recursos necesarios para el ejercicio.

Durante la configuración se asignaron los permisos necesarios para poder administrar los recursos utilizados durante la práctica. Antes de iniciar, se estableció trabajar con un presupuesto máximo de 5.00 USD, el cual se realizó con la herramienta de budget de AWS.



2.2 Creación de la llave de acceso para EC2

Para poder conectarse posteriormente al servidor EC2 mediante SSH se creó una llave de acceso.

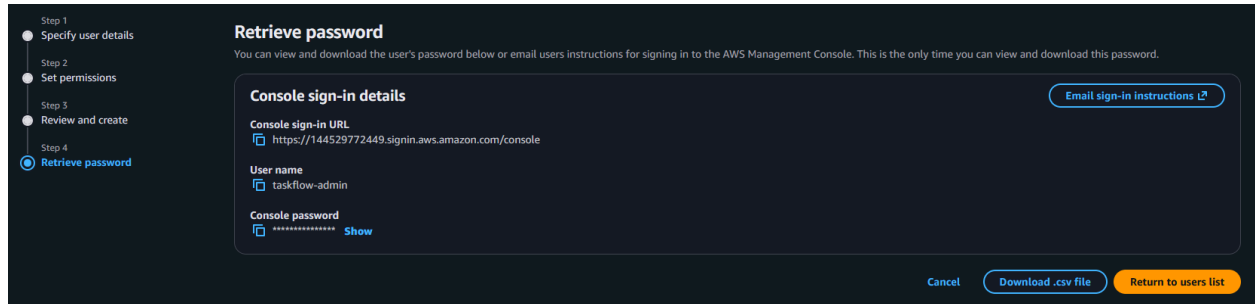
AWS permite generar un par de llaves compuesto por:

- Una llave pública, que queda asociada a la instancia.
- Una llave privada, descargada por el usuario como archivo `.pem`.

En este ejercicio la llave privada se guardó localmente con el nombre:

`taskflow-key.pem`

Esta llave es necesaria para autenticar la conexión SSH al servidor. Por seguridad, la llave privada debe mantenerse protegida y no debe compartirse públicamente.



3. Creación de la instancia EC2

Posteriormente se creó una instancia de Amazon EC2 para funcionar como servidor de la aplicación.

EC2 permite disponer de una máquina virtual en la infraestructura de AWS sin necesidad de contar físicamente con un servidor propio.

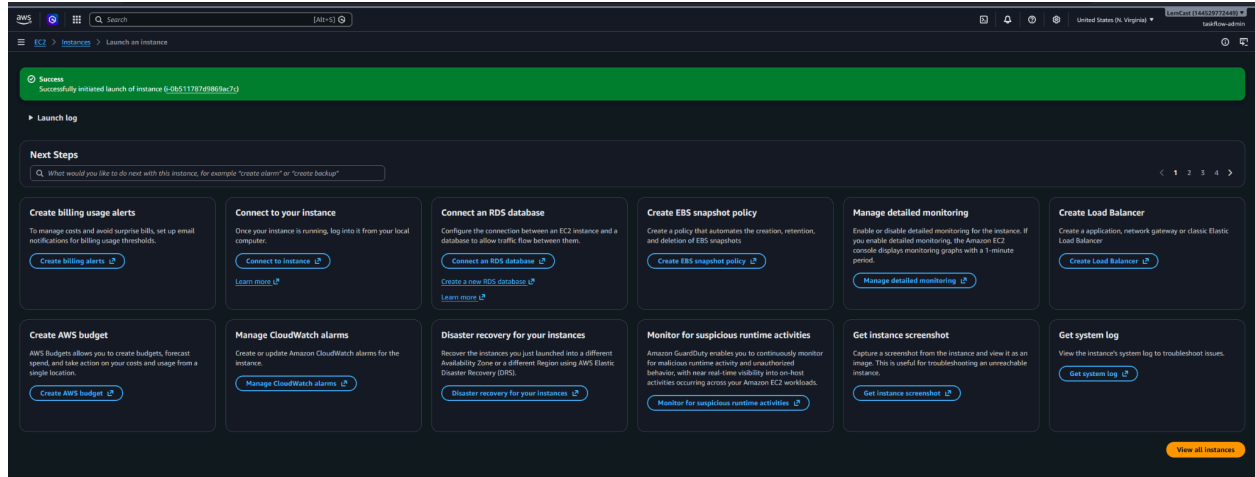
Para la práctica se utilizó una instancia basada en Amazon Linux 2023. La instancia recibió una dirección IP pública que posteriormente se utilizó para conectarse mediante SSH y para acceder a la aplicación desde un navegador.

También se configuró el grupo de seguridad asociado a la instancia para permitir las conexiones necesarias.

Entre los puertos utilizados se encuentran:

- **22/TCP**: utilizado para la conexión SSH.
- **8080/TCP**: utilizado posteriormente por la aplicación Spring Boot.

La IP pública establecida fué 54.87.12.162



4. Conexión al servidor mediante SSH

Una vez creada la instancia EC2, se utilizó Git Bash en el equipo local para establecer una conexión SSH con el servidor.

La llave privada se encontraba dentro de una carpeta local denominada **key**.

Primero se establecieron los permisos adecuados para la llave:

```
chmod 400 taskflow-key.pem
```

Posteriormente se realizó la conexión mediante:

```
ssh -i taskflow-key.pem ec2-user@54.87.12.162
```

Donde:

- **ssh** inicia una conexión mediante el protocolo Secure Shell.
- **-i** indica la llave privada que será utilizada para autenticarse.
- **taskflow-key.pem** corresponde a la llave privada.
- **ec2-user** es el usuario utilizado para acceder a Amazon Linux.
- **54.87.12.162** corresponde a la dirección pública asignada a la instancia EC2.

Una vez establecida la conexión, la terminal mostró el prompt:

Aaron Ulises Garza Rangel

```
[ec2-user@ip-54-87-12-162 ~]$
```

Esto confirmó que se estaba trabajando directamente dentro de la instancia EC2.

[illegible]

5. Verificación del servidor

Una vez dentro de la instancia EC2 se realizaron algunas verificaciones básicas para conocer el entorno disponible.

Se utilizó:

```
uname -a
```

Este comando permite consultar información relacionada con el sistema operativo y el kernel utilizado por el servidor.

También se consultó la dirección IP pública mediante:

```
curl ifconfig.me
```

Finalmente, se verificó la memoria disponible:

free -m

El comando **free -m** muestra información sobre la memoria RAM disponible, utilizada y libre, expresada en megabytes.

Esta verificación fue importante debido a que la instancia utilizada para la práctica cuenta con recursos limitados. Por esta razón, el proyecto se compiló posteriormente en el equipo local en lugar de realizar el proceso de compilación directamente en EC2.

```
1 ssh -t ec2-user@ip-172-31-223-123.us-east-2.compute-1.amazonaws.com
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

6. Instalación de Java

Debido a que la aplicación está desarrollada en Java y utiliza Spring Boot, fue necesario instalar el entorno de ejecución correspondiente en el servidor.

Se instaló Java 21 mediante Amazon Corretto:

```
sudo dnf install -y java-21-amazon-corretto-headless
```

Después se verificó la instalación mediante:

```
java -version
```

El resultado confirmó que Java 21 estaba disponible en la instancia.

La instalación de Java permite que el servidor pueda ejecutar posteriormente el archivo `.jar` generado por el proyecto.

```
complete.  
[ec2-user@ip-172-31-21-223 ~]$ java -version  
openjdk version "21.0.12.1" 2026-08-18 LTS  
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)  
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)  
[ec2-user@ip-172-31-21-223 ~]$ |
```

7. Compilación de la aplicación Maven

Debido a las limitaciones de recursos de la instancia EC2, la aplicación no se compiló directamente en el servidor.

El proyecto se encontraba localmente dentro de la carpeta:

academyMty/taskflow-api

Desde esta ubicación se ejecutó:

```
mvn -q -DskipTests package
```

Este comando utiliza Maven para construir el proyecto y generar el artefacto ejecutable de la aplicación.

La opción:

```
-DskipTests
```

indica que durante este proceso no se ejecutarán las pruebas automatizadas.

Después de completar el proceso de compilación se verificó la existencia del archivo `.jar` generado:

```
ls -lh target/*.jar
```

El archivo generado se encontraba dentro de la carpeta:

8. Transferencia del archivo JAR a EC2

Una vez generado el archivo `.jar`, se utilizó SCP (Secure Copy Protocol) para transferirlo desde el equipo local hacia la instancia EC2.

La transferencia se realizó mediante:

```
scp -i ../../key/taskflow-key.pem target/taskflow-api-*.jar ec2-user@<IP-PÚBLICA>:~/taskflow-api.jar
```

En este caso, la ruta:

```
../../key/taskflow-key.pem
```

se utilizó debido a la estructura de carpetas del proyecto.

El archivo fue enviado al directorio principal del usuario `ec2-user` dentro del servidor y se renombró como:

```
taskflow-api.jar
```

De esta manera, el flujo utilizado fue:

Equipo local

|

| Maven

▼

taskflow-api.jar

|

| SCP

▼

Servidor EC2

|

▼

~/taskflow-api.jar

El proyecto completo no fue enviado ni compilado en EC2. Únicamente se transfirió el artefacto ejecutable generado localmente.

9. Verificación de la integridad del archivo

Después de transferir el archivo al servidor, se verificó que el archivo recibido fuera exactamente el mismo que el generado localmente.

En el equipo local se obtuvo el hash SHA-256 mediante:

```
shasum -a 256 target/taskflow-api-*.jar
```

Posteriormente, dentro de EC2 se calculó el hash del archivo recibido:

```
sha256sum ~/taskflow-api.jar
```

Los valores obtenidos fueron comparados.

Si ambos hashes coinciden, significa que el archivo transferido no sufrió modificaciones durante el proceso de transferencia.

Este procedimiento permite comprobar la integridad del artefacto antes de ejecutarlo en el servidor.

[illegible]

10. Ejecución de la aplicación en EC2

Una vez que el archivo `.jar` se encontraba correctamente en el servidor, se inició la aplicación mediante:

```
nohup java -jar taskflow-api.jar > app.log 2>&1 &
```

El comando realiza varias acciones:

- `java -jar taskflow-api.jar` ejecuta la aplicación Spring Boot.
- `nohup` permite que el proceso continúe ejecutándose aunque la sesión SSH se cierre.
- `> app.log` redirige la salida de la aplicación hacia el archivo `app.log`.
- `2>&1` redirige también los mensajes de error hacia el mismo archivo.
- `&` ejecuta el proceso en segundo plano.

Para observar los mensajes generados durante el inicio de la aplicación se utilizó:

```
tail -f app.log
```

Durante el arranque se esperó encontrar un mensaje similar a:

```
Started TaskflowApiApplication
```

La aparición de este mensaje indicó que la aplicación Spring Boot había iniciado correctamente.

11. Acceso a la aplicación mediante Swagger

Con la aplicación ejecutándose en EC2, se accedió a Swagger desde un navegador web utilizando la dirección pública de la instancia:

```
http://54-87-12-162:8080/swagger-ui/index.html
```

El puerto `8080` corresponde al puerto utilizado por la aplicación Spring Boot.

Para permitir este acceso fue necesario que el Security Group asociado a la instancia tuviera una regla de entrada para:

TCP 8080

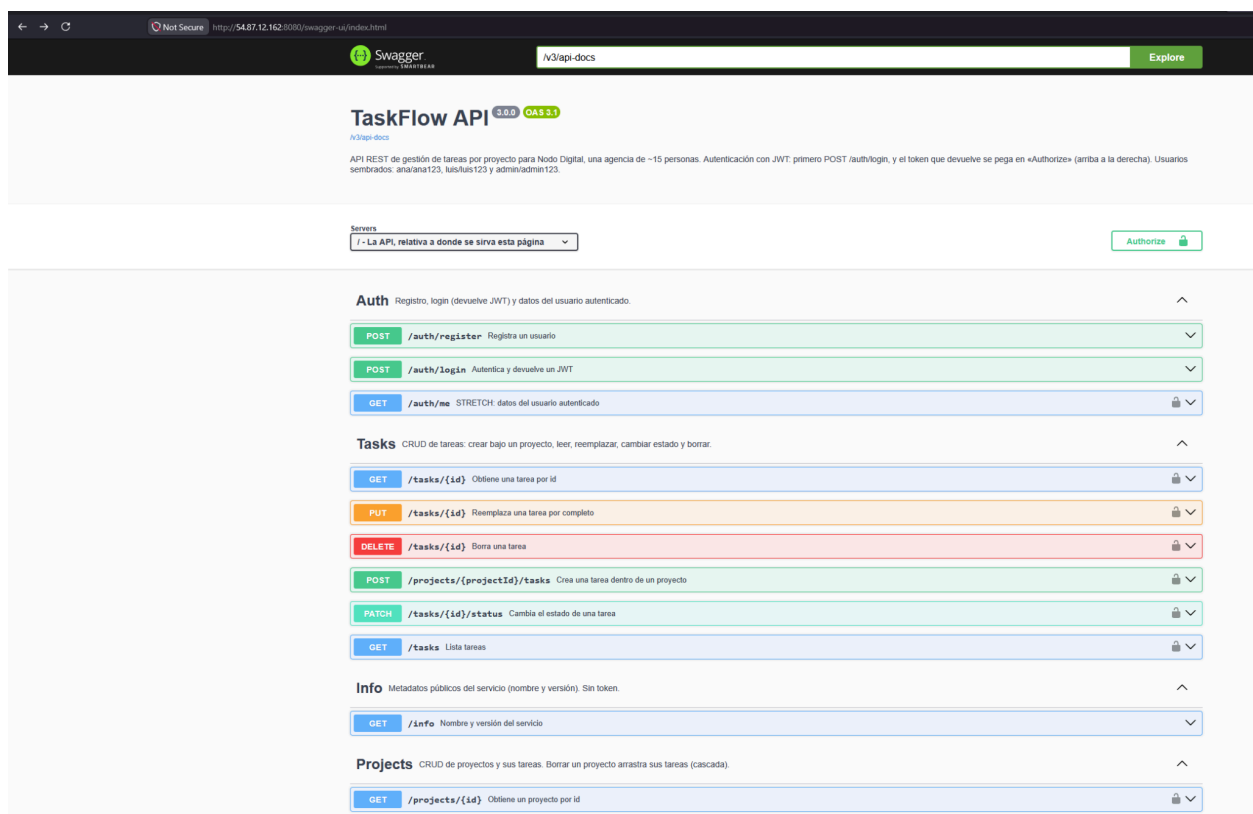
con el origen correspondiente a la configuración indicada durante la práctica.

Si el servidor responde internamente pero la aplicación no es accesible desde el navegador, una de las primeras verificaciones consiste en revisar las reglas de entrada del Security Group.

También se realizó una prueba interna desde EC2 mediante:

```
curl -s localhost:8080/swagger-ui/index.html | head -3
```

Esta prueba permite comprobar que la aplicación está respondiendo desde el propio servidor.



12. Prueba de la API

Una vez que Swagger estuvo disponible desde el navegador, se realizaron pruebas sobre los endpoints de la aplicación.

Se utilizó el usuario:

Usuario: ana

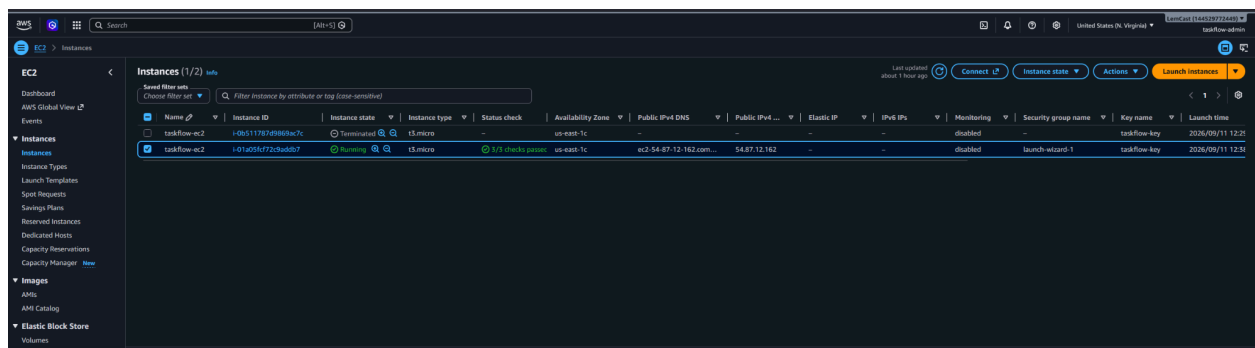
Contraseña: ana123

Después de iniciar sesión se obtuvo el token de autenticación correspondiente.

El token fue utilizado mediante la opción **Authorize** disponible en Swagger para autenticar las peticiones posteriores.

Finalmente, se utilizó uno de los endpoints de la aplicación para crear una tarea.

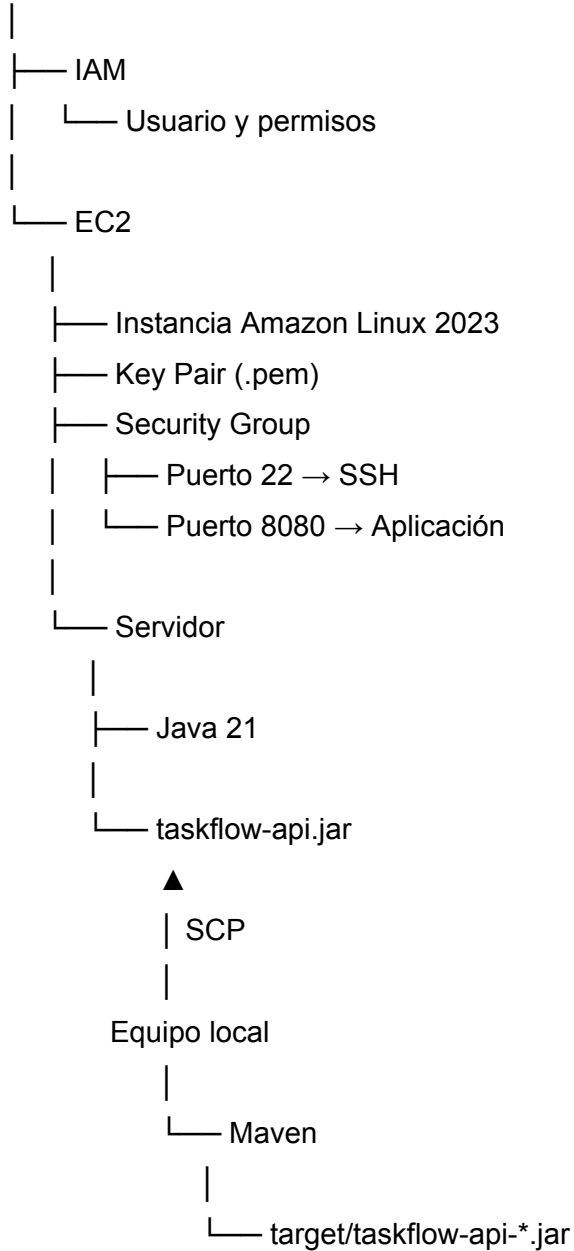
Esta prueba permitió comprobar no solamente que el servidor EC2 estaba accesible, sino también que la aplicación Spring Boot estaba funcionando correctamente y que sus funcionalidades principales podían utilizarse desde el exterior.



13. Flujo completo del despliegue

El proceso realizado puede resumirse de la siguiente manera:

AWS



De esta manera, el proyecto pasó de un entorno local a un servidor accesible públicamente mediante la infraestructura de AWS.

14. Configuración del pipeline de despliegue en AWS

Como complemento al despliegue manual realizado anteriormente, se configuró un pipeline de integración y despliegue continuo utilizando diferentes servicios de AWS.

El objetivo de esta segunda parte fue automatizar el proceso para que, después de realizar cambios en el código fuente y enviarlos al repositorio de GitHub, AWS pudiera detectar el cambio, construir la aplicación y desplegar automáticamente una nueva versión en la instancia EC2.

Los principales servicios utilizados fueron:

- **Amazon DynamoDB:** almacenamiento de información relacionada con eventos de la aplicación.
- **Amazon S3:** almacenamiento de artefactos utilizados por el pipeline.
- **AWS CodeBuild:** compilación automática del proyecto.
- **AWS CodeDeploy:** despliegue de la aplicación en EC2.
- **AWS CodePipeline:** coordinación de las diferentes etapas del proceso.
- **IAM:** administración de los roles y permisos necesarios para que los servicios pudieran comunicarse entre sí.

15. Configuración previa y rol de CodeDeploy

Durante la configuración inicial se encontró una restricción relacionada con el tipo de cuenta de AWS utilizada. Las cuentas tradicionales mostraban una pantalla indicando que era necesario completar la configuración de la cuenta antes de poder utilizar determinados servicios.

Para continuar con la práctica se utilizó la nueva experiencia de configuración de cuenta disponible mediante settings.aws.com, trabajando en la región de **Virginia (us-east-1)**, donde estaban disponibles los servicios requeridos para la actividad.

Entre estos servicios se encontraban:

- AWS CodeDeploy
- AWS CodePipeline
- AWS CodeBuild

Para permitir que CodeDeploy pudiera interactuar con los recursos necesarios se creó un nuevo rol IAM.

La configuración utilizada fue:

Servicio: AWS CodeDeploy

Rol: taskflow-codedeploy-role

Política: AWSCodeDeployRole

El rol proporciona las credenciales y permisos necesarios para que CodeDeploy pueda administrar el proceso de despliegue sobre la instancia EC2.

16. Configuración de la aplicación en CodeDeploy

Dentro de AWS CodeDeploy se creó una aplicación con los siguientes parámetros:

Nombre: taskflow

Plataforma de cómputo: EC2/On-premises

Esta aplicación representa el proyecto que será administrado mediante CodeDeploy.

Posteriormente se creó el grupo de despliegue:

taskflow-dg

Aaron Ulises Garza Rangel

Se utilizó el rol IAM creado anteriormente:

`taskflow-codedeploy-role`

El tipo de despliegue seleccionado fue:

In-place

Este tipo de despliegue permite actualizar la aplicación directamente sobre una instancia existente en lugar de crear una instancia nueva.

Para identificar la instancia EC2 que recibiría el despliegue se utilizó una etiqueta:

Key: Name

Value: `taskflow-ec2`

También se indicó que el agente SSM no debía instalarse nuevamente, debido a que ya se encontraba disponible como parte de la configuración previa.

La configuración de despliegue utilizada fue:

`CodeDeployDefault.AllAtOnce`

Finalmente, no se habilitó el balanceador de carga debido a que la práctica utiliza una única instancia EC2.

17. Creación del pipeline mediante CodePipeline

Una vez configurado CodeDeploy, se creó un pipeline mediante AWS CodePipeline.

El pipeline se configuró utilizando la opción:

Build custom pipeline

El nombre asignado fue:

taskflow-pipeline

El modo de ejecución seleccionado fue:

Queued

AWS creó automáticamente el rol de servicio correspondiente:

AWSCodePipelineServiceRole-us-east-2-taskflow-pipeline

Para almacenar los artefactos generados durante el proceso se utilizó un bucket de Amazon S3 personalizado:

taskflow-artefactos-<usuario>

18. Configuración de la etapa Source

La primera etapa del pipeline corresponde a la obtención del código fuente.

Se configuró GitHub como proveedor mediante GitHub App.

Los parámetros utilizados fueron:

Repositorio: taskflow-aws-<usuario>

Rama: main

También se habilitaron los eventos de webhook para iniciar automáticamente el pipeline cuando se realizara un cambio en el repositorio.

La opción utilizada fue:

Start your pipeline on push and pull request events

De esta manera, un **git push** al repositorio podía iniciar automáticamente una nueva ejecución del pipeline.

19. Configuración de la etapa Build mediante CodeBuild

La segunda etapa del pipeline correspondió a la construcción del proyecto.

Para ello se creó un proyecto de AWS CodeBuild denominado:

`taskflow-build`

Se utilizó una imagen administrada por AWS basada en Amazon Linux, con el runtime Standard versión 6.0.

La configuración del proyecto indicó que debía utilizarse un archivo `buildspec.yml` localizado en la raíz del repositorio.

El archivo `buildspec.yml` contiene las instrucciones que CodeBuild debe seguir para construir el proyecto.

De esta manera, el proceso de compilación que anteriormente se realizaba manualmente mediante Maven en el equipo local pasó a poder ejecutarse automáticamente dentro de CodeBuild.

20. Configuración de la etapa Test

La etapa independiente de pruebas de CodePipeline fue omitida mediante la opción:

`Skip test stage`

La razón fue que las pruebas automatizadas del proyecto ya se ejecutaban como parte del proceso de construcción definido para Maven.

Por lo tanto, no fue necesario crear una etapa adicional exclusivamente para pruebas dentro de CodePipeline.

El flujo quedó simplificado como:

Source → Build → Deploy

21. Configuración de la etapa Deploy

La última etapa del pipeline se configuró utilizando AWS CodeDeploy.

Los parámetros principales fueron:

Proveedor: AWS CodeDeploy

Input Artifact: BuildArtifact

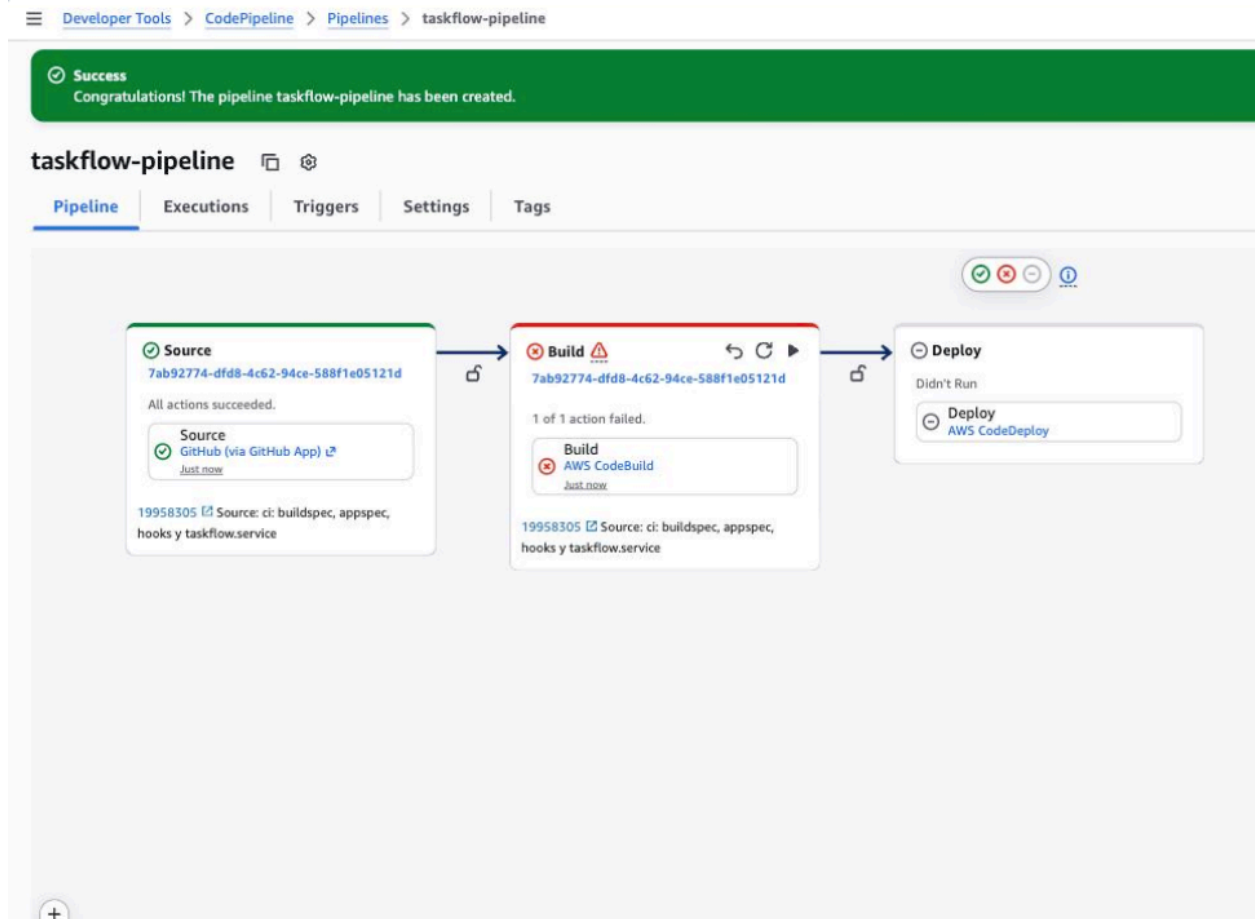
Aplicación: taskflow

Deployment Group: taskflow-dg

El artefacto generado durante la etapa de construcción se utilizó como entrada para CodeDeploy.

El rollback automático se dejó desactivado para los fines de la práctica.

Una vez terminada la configuración, el pipeline quedó preparado para recibir cambios desde GitHub, construir la aplicación y desplegarla automáticamente en EC2.



22. Configuración y uso de DynamoDB

Como parte complementaria de la actividad se trabajó con Amazon DynamoDB para almacenar información relacionada con los eventos de la aplicación.

DynamoDB es un servicio de base de datos NoSQL administrado por AWS. A diferencia de una base de datos relacional tradicional, utiliza tablas compuestas por elementos y atributos y permite definir claves para identificar los registros.

Durante la práctica se utilizó una tabla denominada:

taskflow-eventos

La configuración y los comandos empleados se documentaron en:

evidencia/comandos-dynamodb.md

También se incluyó documentación adicional sobre el modelo de datos utilizado en:

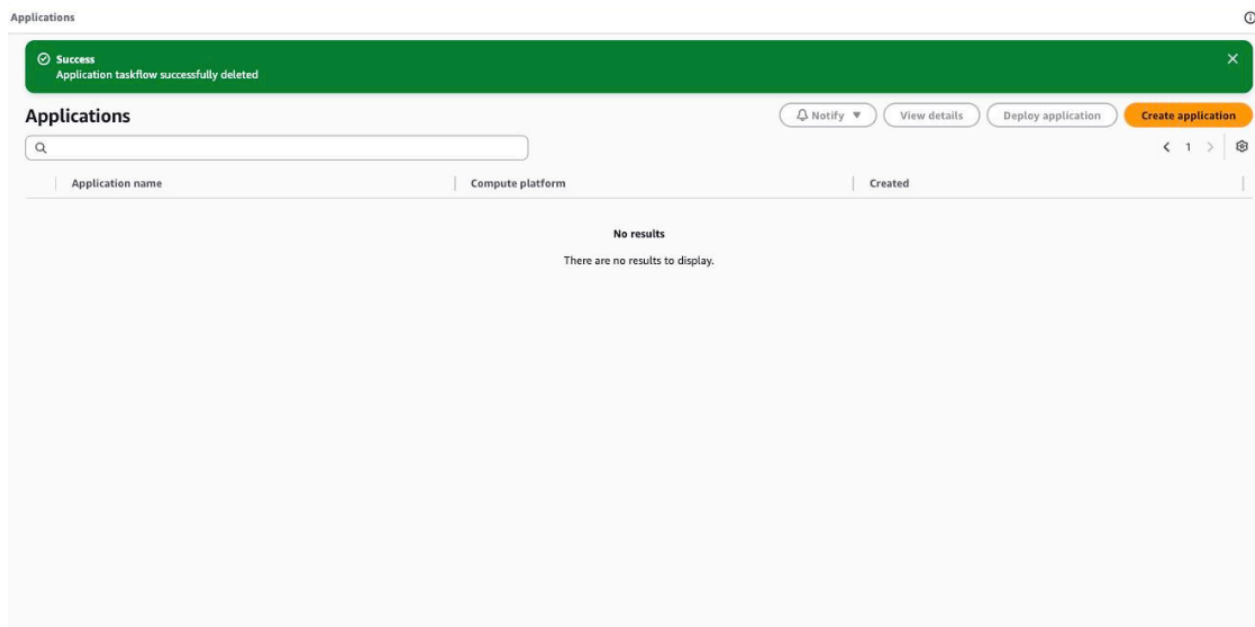
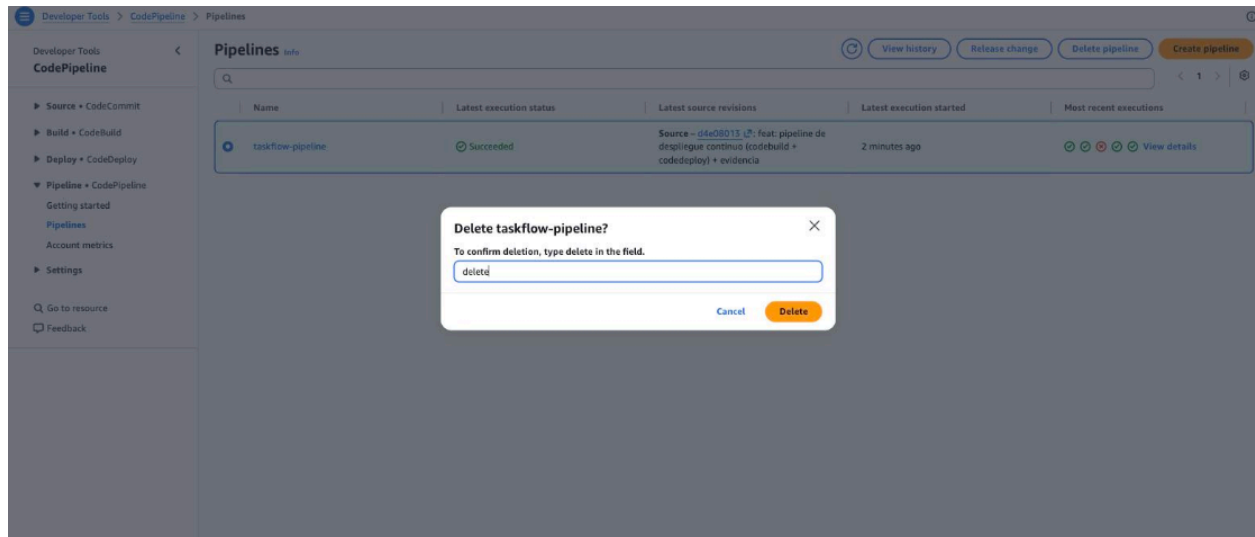
evidencia/modelado.md

El uso de DynamoDB permitió complementar el ejercicio de despliegue con una introducción al uso de servicios administrados de almacenamiento dentro de AWS.

23. Limpieza de los recursos de AWS

Una vez finalizada la práctica, se realizó la limpieza de los recursos creados para evitar consumos innecesarios y posibles cargos posteriores.

La eliminación se realizó siguiendo un orden que evitara dejar recursos dependientes entre sí.



24. Conclusión del proceso de automatización

La segunda parte de la actividad permitió evolucionar desde un despliegue manual hacia un proceso automatizado de integración y despliegue continuo.

Inicialmente, la aplicación se construía manualmente en el equipo local y el archivo `.jar` era transferido mediante SCP hacia la instancia EC2. Posteriormente, mediante CodePipeline, CodeBuild y CodeDeploy, este proceso pudo automatizarse.

El flujo final quedó establecido de la siguiente manera:

Desarrollador

|

| git push

▼

GitHub

|

▼

CodePipeline

|

|— Source

|

|— Build → CodeBuild → Maven

|

|— Deploy → CodeDeploy

|

▼

EC2

|



Taskflow API

Además de comprobar el funcionamiento normal del pipeline, se realizaron pruebas de errores controlados. Esto permitió observar cómo AWS reporta problemas relacionados con permisos de S3 y errores durante los hooks de CodeDeploy.

La práctica permitió comprender de forma integral el recorrido que realiza una aplicación desde el repositorio de código fuente hasta su ejecución en un servidor, así como la importancia de los permisos IAM, los artefactos, los logs y los mecanismos de diagnóstico dentro de un proceso de CI/CD.

Finalmente, la eliminación de los recursos utilizados permitió cerrar la práctica de manera segura y evitar mantener infraestructura innecesaria en la cuenta de AWS.